

cVars

Camera Vision Automated Recycling Sorter

Melody Kuoch and Jaimen Whittenburg

As 2027's Summer Interns, we were tasked with simulating a GEEN 1400 freshman project– creating an automated recycling system. We were encouraged to note our trials and tribulations to eventually culminate to a comprehensive end-to-end guide for engineering support staff to reference.

Topics Covered
Machine Learning & Computer Vision
Raspberry Pi Setup & Python Environment
Motor Control & GPIO Wiring
3D Printed Mechanical Design
System Integration
Common Struggles & Debugging Strategies

Table of Contents

1. [Project Overview:](#)
2. [System Architecture](#)
3. [Bill of Materials](#)
4. [Machine Learning and Dataset](#)
5. [RPi Set up](#)
6. [Python Environment and Ultralytics](#)
7. [GPIO Wiring and Pin Assignments](#)
8. [Motor Control \(TB6612FNG\)](#)
9. [Servo Control](#)
10. [Ultrasonic Sensor](#)
11. [Mechanical Design](#)
12. [System Integration](#)
13. [Common Struggles and Debugging](#)
14. [Student Q&A Reference](#)
15. [Scripts](#)

Project Overview:

The Camera Vision Automated Recycling Sorter is an electromechanical system that autonomously identifies recyclable material using a trained machine learning model and physically sorts them into the correct bin without human intervention. When we were first introduced to the three summer technical project options (bike generator, self watering plant, and camera vision recycling) both me and Jaimen gravitated towards the camera vision project because it felt like a challenging and new experience for us both. That being said, we are both at the end of our academic careers. This is to say, this project is not for the person with zero experience on the software side of things. It was extremely challenging and required a level of confidence in the world of engineering to not be discouraged. My biggest personal advice from a mental standpoint is to really really assure students that the turbulence of this project may be heavier than others and more support may be needed and that is ok! Once that is made clear the next biggest hurdle is coming to a clear understanding of the scope and work flow of the system.

Overall, the goal of this project is to be able to accurately sort recyclable materials using camera vision. There are two main systems that must work together in order for this project to be successful, which are the software system and the mechanical system. The software needs to be able to use a camera and be able to use some sort of camera vision library to be able to classify the material. After the material is classified there needs to be some sort of mechanical system that can actually physically sort the material. These two systems need to be able to communicate with each other effectively in order for the system to be successful. Neither of these systems are easy to fabricate, let alone get them to communicate and work together. The goal of this document is to provide a guidebook on how to be successful with this project.

System Architecture:

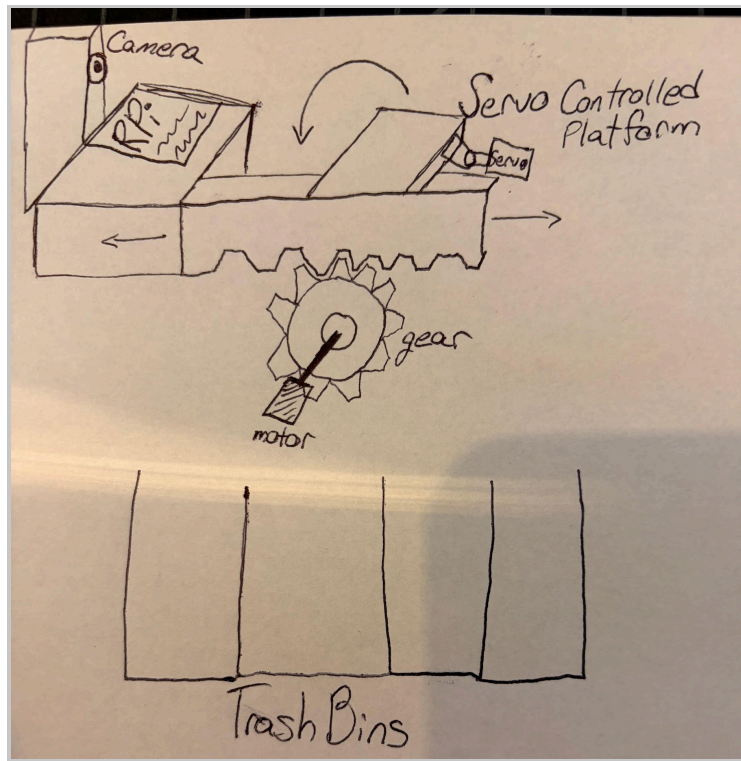
There were a few options we discussed in our early brainstorm. Initially we thought of sorting materials purely on physical qualities. For instance, weight, conductivity, shape etc. This option is definitely tangible; however, pretty early on we realized although the least software heavy, it would be both inaccurate and extensive to compile all these different material checks. Eventually we found ourselves thinking of some sort of machine learning model that could categorize materials. As machine learning, flock cameras, AI, databases grow in the general social consciousness, I presume this route will become more and more popular amongst freshman engineering students.

The next question we asked ourselves was how. Me nor Jaimen have touched ML or CV and combining the two tools felt abstract. High level, we knew we wanted a camera to take a picture, the picture would then be run through a MLM, then as a result of this, a material label output would lead to some sort of physical sorting mechanism. That iffy area between a camera taking a picture and a material label received in addition to what physical sorting route to choose was blurry. Hopefully by the end of this guide our decisions become a bit clearer, but eventually this is how we decided our system to work:

1. An item is placed in front of a Pi Camera V3
2. The user presses a physical button to trigger a scan
3. A photo is captured and passed through the trained YOLO classification model
4. If confidence exceeds 70% the material label is determined
5. If confidence is below 70% the material label defaults to trash

6. A DC Motor moves a platform along a rack and pinion to the correct bin position, guided by an ultrasonic sensor
7. A servo motor actuates a 3D printed trap door to deposit the item
8. The platform returns to the home position ready for the next item

Here is a sketch of our system showing our idea of how it would all work together.



Your sketch does not have to be perfect but do your best to outline your critical components and what their purpose is.

This is the process we decided on however there are other components that can be successfully used. There are tons of various different mechanical designs that achieve the task of sorting these recyclables, but most mechanical designs that we thought through needed one main drive motor with additional servo motors. Deciding on what motor to use for your project can be a daunting task because there are so many different types

of motors. Three essential types of motors are direct current motors, servo motors, and stepper motors. Below is a chart of what each of these motors do and where they are applicable in this project.

Motor Type	Description	Extra Components Needed	Uses in This Project Scope
DC Motor	Simplest of the three — apply voltage, it spins, reverse polarity, it spins the other way Great for continuous rotation tasks: fans, wheels, pumps Downside: no built in precise control over exact position without adding extra sensors	In order to control the motor you will need to buy a motor driver. I have found much success with the SparkFun Dual TB6612FNG Motor Driver, it can control two DC motors at once.	Can be used as the main drive motor. In our case we chose to use a 6V DC Motor to drive our gear box. This came at a cost as we had to also set up a separate sensor to control the position.
Servo Motor	A DC motor + gearbox + feedback (potentiometer or encoder) + control circuitry, all bundled together You tell it a specific angle (usually via PWM signal), and it moves there and holds that position Typically limited range of motion (0-180° for hobby servos), though continuous-rotation servos exist Great for precise positioning: robot arms, steering mechanisms, camera gimbals Closed-loop control — it actively corrects itself if something pushes it off position	No extra components needed besides a microcontroller (like arduino uno)	Servo motors worked great for small movements. In our case we used it to tip our platform to dump off the recyclables. Super useful to be able to set exact angles for the motor to move to.
Stepper Motor	Moves in discrete fixed increments ("steps") — like 1.8° per step — rather than continuously You don't tell it an angle directly; you tell it "take N steps" and it counts them off Open-loop by default (no feedback) — it assumes it moved correctly unless it stalls/skips steps Great for precise, repeatable positioning without	Stepper motors also need a dedicated motor driver, I would recommend looking through SparkFun's motor drivers as they are a local company that you can drive to if needed.	Super useful for a drive motor. Able to precisely move at fixed increments. This honestly would have been a better choice to drive our gear box than the DC Motor was, but we were

	needing a feedback sensor: 3D printers, CNC machines, camera sliders		unable to implement it due to time constraints. I would recommend using a stepper motor over a DC motor if you are using the motor for any sort of positioning task.
--	--	--	--

Following the completion of this project, hopefully students will gain experience on

- Training and deploying a real machine learning classification model
- Raspberry Pi GPIO control for motors and sensors
- Python scripting for hardware integration
- 3D CAD design and iterative manufacturing
- System-level thinking: connecting software, electronics, and mechanics

Subsystem	Component	Role
Vision	Pi cam	Captures images for classification
Intelligence	YOLO11 + best.pt	Classifies material type
Control	Rpi 4	Central controller, runs Python scripts
Motor Driver	SparkFun TB6612FNG	Controls DC motor direction
Linear Motion	DC Motor + Rack & Pinion	Moves platform to correct bin
Door Control	Servo	Actuates 3D Printed trap door
Positioning	HC-SR04 Ultrasonic Sensor	Measures platform distance
User Input	Push Button	Triggers scan cycle
Power (Pi)	5V USB Power Bank	Powers pi,cam, servo, GPIO
Power (motor)	6V AA Battery Pack	Powers DC motor via TB6612FNG VM pin
Storage	External WD Drive (ExFat)	Stores Python Packages (ultralitics)

Bill of Materials

DC Motor	\$6	Can checkout with an Electronic center engineer
Servo Motor	\$4	Can checkout with an Electronic center engineer
Bearings x10	\$7	Free in project vault
3D Printer Filament	\$10	Free after attending 3dPrinter Workshop
Laser Cutter Material	\$5	Can buy at ITLL launch point

Ultrasonic Sensor	\$7	Can checkout with an Electronic center engineer
SparkFun Motor Driver - Dual TB6612FNG	\$15	Can checkout with an Electronic center engineer
Raspberry Pi 4	\$124	Can checkout with an Electronic center engineer
Double A Batteries	\$15	May need to purchase outside of ITLL
External Hard drive	~\$50	May need to purchase outside of ITLL
Total:	\$243	

Machine Learning and Dataset:

Choosing a task type

Engineers should guide students to understand the difference between classification and object detection before they start sourcing data.

Approach	Pros	Cons	Best for this project
Image classification (what we used)	Simple dataset structure, faster training	Assumes one item fills the screen	YES
Object detection	Localizes items with	More complex data	Not needed

	bounding boxes	structure, unnecessary for the scope of our project	
Manufacturer API (e.g Google Vision or Claude API)	No training needed, fast to implement	Costs associated, latency, less learning	Not recommended

Dataset sourcing

Students may struggle finding a usable data set. These are the options I considered

1. Kaggle ([kaggle.com](https://www.kaggle.com)) — search “recycling classification”. Datasets are preorganized into class folders.
Perfect for YOLO classification
2. Roboflow UNiverse (universe.roboflow.com) — good for object detection data sets with bounding boxes. Requires account, Download YOLOv8 format
3. Custom Photos — have students photograph their own items. Time-consuming but produces data most relevant to their specific environment and needs

Trouble shooting: api download dataset

WARNING: Roboflow datasets downloaded without the API often fail through the browser. The API Method (pip install roboflow) is simpler but may feel out of scope for some students. Kaggle direct download is the most reliable path for beginners.

Dataset Classes Used

Class	Training images	Notes
Cardboard	1,128	Boxes, paper packaging
Glass	1388	Bottles, jars
Metal	744	Cans, foil
Paper	1068	Documents, newspaper
Plastic	1277	Bottles, bags, containers
Trash	361	General waste –fewest images
TOTAL	5966	Split 80% Train 15% Val 5% test

RPi Setup + Python Environment and Ultralytics:

Done on student computer or laptop

Script 1: split data into three buckets—train/val/test

Python
<pre>import os, shutil, random classes = ['cardboard', 'glass', 'metal', 'paper', 'plastic', 'trash'] source = 'archive/original' random.seed(42)</pre>

```

for cls in classes:
    images = os.listdir(f'{source}/{cls}')
    random.shuffle(images)
    n = len(images)
    splits = {
        'train': images[:int(n*0.8)],
        'val': images[int(n*0.8):int(n*0.95)],
        'test': images[int(n*0.95):]
    }
    for split, files in splits.items():
        os.makedirs(f'dataset/{split}/{cls}', exist_ok=True)
        for f in files:
            shutil.copy(f'{source}/{cls}/{f}', f'dataset/{split}/{cls}/{f}')

```

Script 2: train the model

Python

```

from ultralytics import YOLO

model = YOLO('yolo11n-cls.pt') # downloads pretrained weights automatically

model.train(
    data='dataset',
    epochs=10,
    imgsz=224,
    batch=16,
    name='recycling_classifier'
)

```

Done on Pi terminal

Install 3: enable camera—install cam library

```
bash
```

```
Sudo apt-get install python3-picamera2 -y
```

3. Test

```
bash
```

```

rpicalm -hello
*note* should open a camera window quickly and close

```

Install 4: file transfer—External Drive (ExFat formatted)

bash

```
cp /media/<username>/ExtraSpace/best.pt ~/recycling-detector/  
cp /media/<username>/ExtraSpace/sorter_web.py ~/recycling-detector/
```

Troubleshooting: python download storage error

WARNING: This is the most common point of failure for students. The Pi's 16GB SD card cannot hold the full Ultralytics/PyTorch install by default because pip installs GPU-targeted NVIDIA packages (CUDA, cuDNN) that total over 2GB — none of which the Pi needs since it has no NVIDIA GPU.

4.1: create directories in external drive

Bash

```
mkdir -p /media/<username>/ExtraSpace/packages  
mkdir -p /media/<username>/ExtraSpace/tmp
```

4.2: set temp and cache directories to the external drive

Bash

```
export TMPDIR=/media/<username>/ExtraSpace/tmp  
export PIP_CACHE_DIR=/media/<username>/ExtraSpace/tmp
```

4.3: install ultralytics to the external drive

Bash

```
pip install ultralytics \  
--target=/media/<username>/ExtraSpace/packages \  
--cache-dir=/media/<username>/ExtraSpace/tmp \  
--extra-index-url https://download.pytorch.org/whl/cpu \  
--break-system-packages
```

4.4: run scripts with PYTHONPATH set for any future compilations that require the use of ultralytics

Bash

```
PYTHONPATH=/media/<username>/ExtraSpace/packages python3 sorter_web.py
```

note

- <username> is a place holder use your credentials and don't add the <>
- sorter_web.py is just an example. Use your own file name here!
- ExtraSpace may be the incorrect mount name use `bash lsblk` to figure out the name of your external drive if compiling fails to access the external drive
- Only use this path compiling script if you need ultralytics. Many tests like the [servo.py](#), [motor.py](#), and [button.py](#) tests dont need the path reference so you can just use `python3 <filename>` to compile

Troubleshooting: VENV and Ultralytics download issue on external drive

NOTE: When I first started this project, youtube videos on rpi and ultralytics direct you to use a venv or virtual environment in order to keep the dependencies of ultralytics isolated from the pi's internal packages and prevent corruption between the two. This is not untrue. However, because we had that storage issue and had to use an external drive, this caused another issue. External drives are formatted in ExFAT. ExFAT by a turn of events doesn't support track pointer references which Python's venv requires internally for symlinks(of course it doesn't) The work around here was using that `-target` flag. This keeps the venv on the SD card and installs packages to the external drive.

Troubleshooting: Mount name mismatch

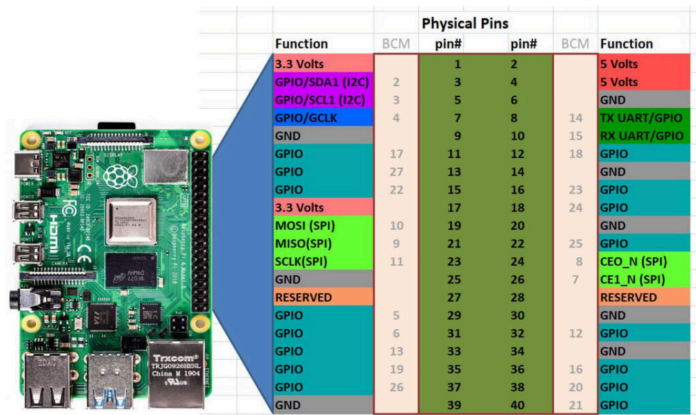
Warning: Another weird issue I ran into was sometimes the Pi would mount the external drive under different names on each boot. To check what the mount name is use command `lsblk` and change the `ExtraSpace` to whatever the mount name is for your drive.

GPIO Wiring and Pin Assignments:

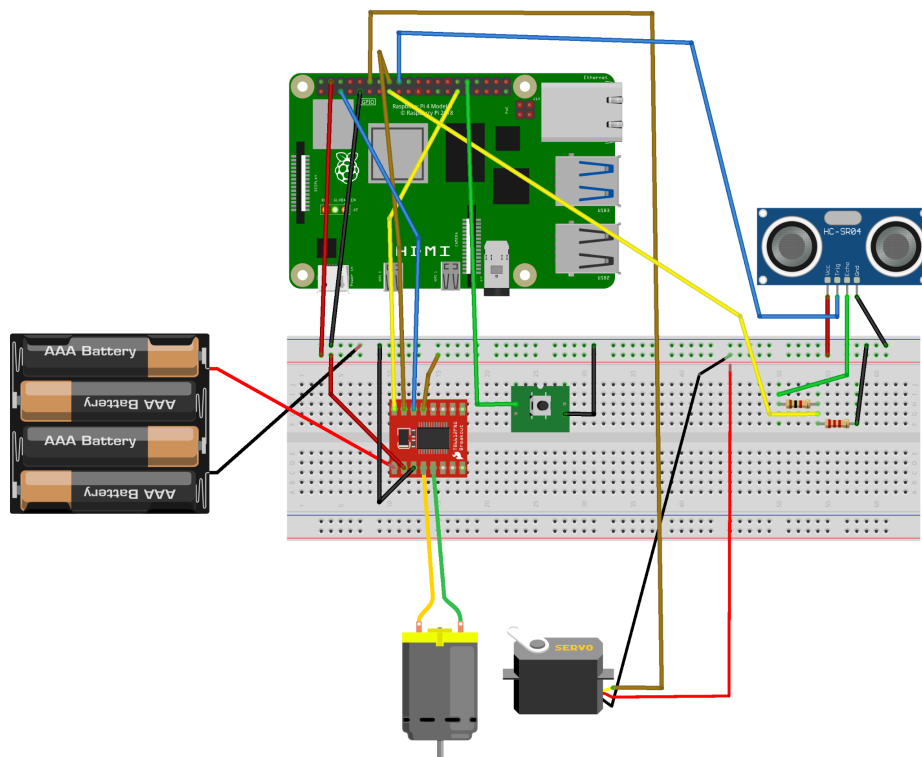
Before getting into the fine details of where each wire goes, it is important to understand some key terms that will come up when wiring up motors and other components to a microcontroller. First off, what does GPIO even mean? GPIO stands for general purpose input/output. This means that these GPIO pins can be used as either an input or an output pin. When using it as an input pin, it reads a digital voltage state, for an RPi this is either 3.3V or 0V, you can think of it like a light switch, it is either on or off, a 1 or a 0, with no inbetween. When using it as an output pin, it sends out a signal of either 3.3V or 0V to control a component. We use software to control these pins, sending off signals to control components, or reading inputs from a sensor.

Another term students wiring this project should be familiar with is PWM (pulse width modulation). This is a method that microcontrollers use to control how much voltage goes to a component. Instead of being only on or off, PWM allows for microcontrollers to send/read voltages within a range, for example anywhere from 0-5V and everything in between. PWM is useful when one needs to be able to control the amount of electricity that is sent to a component.

The Raspberry Pi 4 has forty pins, so wiring can be daunting. Below is a chart of each pin's purpose (please note that the physical pin number does not always line up with the software side GPIO pin. For example GPIO 17 is physical pin 11)



The diagram below shows exactly how our design was wired up:



fritzing

This wiring diagram can look confusing, but the way to look at it is to look at what component you are trying to wire and just follow the wires on the diagram and see where they are going and what they are connected to. Most components need a voltage source, a ground wire, and a couple of other control wires.

Motor Control:

The DC motor is probably the most confusing component to control. Below is a chart on the purpose of each pin on the driver and where they should connect to:

Pin Label	Function	Power/Input/Output	Notes
VM	Motor Voltage	Power	This is where you provide power for the motors (2.2V to 13.5V)
VCC	Logic Voltage	Power	This is the voltage to power the chip and talk to the microcontroller (2.7V to 5.5V)
GND	Ground	Power	Common Ground for both motor voltage and logic voltage (all GND pins are connected)
STBY	Standby	Input	Allows the H-bridges to work when high (has a pulldown resistor so it must actively pulled high)
AIN1/BIN1	Input 1 for channels A/B	Input	One of the two inputs that determines the direction.
AIN2/BIN2	Input 2 for channels A/B	Input	One of the two inputs that determines the direction.
PWMA/PWMB	PWM input for channels A/B	Input	PWM input that controls the speed
A01/B01	Output 1 for channels A/B	Output	One of the two outputs to connect the motor
A02/B02	Output 2 for channels A/B	Output	One of the two outputs to connect the motor

Find code for integration below

Python

```
import RPi.GPIO as GPIO
import time

# — PIN ASSIGNMENTS (BCM GPIO) —
AIN1 = 5 # Physical Pin 29 — motor direction
AIN2 = 23 # Physical Pin 16 — motor direction
PWMA = 13 # Physical Pin 33 — motor speed
# STBY is hardwired to 5V (Physical Pin 2) — always enabled

# — SETUP —
GPIO.setmode(GPIO.BCM)
```

```

for pin in [AIN1, AIN2, PWMA]:
    GPIO.setup(pin, GPIO.OUT)

motor_pwm = GPIO.PWM(PWMA, 1000) # 1000Hz PWM frequency
motor_pwm.start(0)                # start at 0% duty cycle (stopped)

# — FUNCTIONS —
def motor_forward(duration):
    GPIO.output(AIN1, GPIO.HIGH) # AIN1 HIGH + AIN2 LOW = forward
    GPIO.output(AIN2, GPIO.LOW)
    motor_pwm.ChangeDutyCycle(75) # 75% speed — adjust if needed
    time.sleep(duration)
    motor_stop()

def motor_backward(duration):
    GPIO.output(AIN1, GPIO.LOW) # AIN1 LOW + AIN2 HIGH = backward
    GPIO.output(AIN2, GPIO.HIGH)
    motor_pwm.ChangeDutyCycle(75)
    time.sleep(duration)
    motor_stop()

def motor_stop():
    motor_pwm.ChangeDutyCycle(0) # 0% duty cycle = no power
    GPIO.output(AIN1, GPIO.LOW)
    GPIO.output(AIN2, GPIO.LOW)

```

GPIO.setmode(GPIO.BCM) tells the Pi to use BCM (GPIO) pin numbering, not physical pin numbers. Students frequently confuse these — always use BCM numbers in code.

GPIO.PWM(PWMA, 1000) creates a PWM signal on the speed pin at 1000Hz. PWM (Pulse Width Modulation) rapidly switches the pin between HIGH and LOW. The ratio of on-time to off-time (duty cycle) controls how much average voltage the motor receives — higher duty cycle = faster motor.

motor_pwm.start(0) begins the PWM signal at 0% duty cycle, meaning the motor starts stopped. AIN1 HIGH + AIN2 LOW = forward direction. AIN1 LOW + AIN2 HIGH = backward. This is how the H-bridge reverses current flow through the motor.

ChangeDutyCycle(75) sets speed to 75%. Adjust this value between 0-100 depending on how fast the motor needs to move.

motor_stop() sets duty cycle to 0 and both direction pins LOW — this coasts the motor to a stop rather than braking hard.

Troubleshooting: Motor startup issues

WARNING: The DC Motor must be powered by a separate 6V battery pack connected to the VM pin and the ground must be shared (common ground between pi GND and battery GND) as shown by the wiring diagram above

It is also great practice to comment out in your python script the declarations and functions to make troubleshooting a breeze!

Servo Control:

The servo motor is much easier to wire. The servo motor has three wires, a 5V power source, a ground wire, and a control wire. The 5V wire goes to the 5V pin on the RPi, ground to ground, and the control wire goes to a GPIO pin.

Find code below

Python
<pre>import RPi.GPIO as GPIO import time # — PIN ASSIGNMENT (BCM GPIO) — SERVO_PIN = 18 # Physical Pin 12 # — SETUP — GPIO.setmode(GPIO.BCM)</pre>

```

GPIO.setup(SERVO_PIN, GPIO.OUT)

servo = GPIO.PWM(SERVO_PIN, 50) # 50Hz — standard frequency for servos
servo.start(0)          # start with no signal

# — FUNCTIONS —
def door_open():
    servo.ChangeDutyCycle(7.5) # 7.5% = 90 degrees = close position
    time.sleep(1)             # wait for servo to reach position

def door_close():
    servo.ChangeDutyCycle(2.5) # 2.5% = 0 degrees = open position
    time.sleep(1)
    servo.ChangeDutyCycle(0)   # stop signal to prevent jitter

# — CLEANUP —
def cleanup():
    servo.stop()
    GPIO.cleanup()

```

Servos use PWM at 50Hz (50 pulses per second) — this is the industry standard for servo motors and is different from the 1000Hz used for the DC motor.

The duty cycle value determines the angle, not the speed. The relationship is:

- 2.5% duty cycle → 0 degrees (open)
- 12.5% duty cycle → 180 degrees (close)

servo.ChangeDutyCycle(0) after opening is important — it stops the PWM signal entirely, preventing the servo from jittering or drawing unnecessary current while holding its position. Without this, the servo buzzes constantly.

The time.sleep(1) gives the servo time to physically reach the target angle before the script continues. Without it, the next command may fire before the servo finishes moving.

Ultrasonic Sensor:

The way an ultrasonic sensor detects distance is by using soundwaves! The sensor sends out a high frequency sound wave that reflects off surfaces, it uses the time from when the sound wave was sent to when it received the signal back to track the distance. The ultrasonic sensor has 4 pins that need to be connected. It has a 5V power supply that goes to a 5V pin on the RPi, a ground that goes to ground on the RPi, and then two more pins named Echo and Trig. The trig pin sends out the sound wave, while the echo pin measures the time it takes for the wave to return to the sensor. Both the trig and echo pins go to GPIO pins on the RPi.

Python

```
import RPi.GPIO as GPIO
import time

# — PIN ASSIGNMENTS (BCM GPIO) —
TRIG = 17 # Physical Pin 11 — sends sound pulse
ECHO = 27 # Physical Pin 13 — receives echo (NEEDS VOLTAGE DIVIDER)

# — SETUP —
GPIO.setmode(GPIO.BCM)
GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)
GPIO.output(TRIG, GPIO.LOW)
time.sleep(0.5) # let sensor settle on startup

# — FUNCTIONS —
def get_distance():
    """Measure distance from sensor to platform in cm."""
    # Send a 10 microsecond trigger pulse
    GPIO.output(TRIG, GPIO.HIGH)
    time.sleep(0.00001) # 10 microseconds
    GPIO.output(TRIG, GPIO.LOW)

    # Measure how long ECHO pin stays HIGH
    pulse_start = time.time()
    while GPIO.input(ECHO) == GPIO.LOW:
        pulse_start = time.time()

    pulse_end = time.time()
    while GPIO.input(ECHO) == GPIO.HIGH:
```

```

    pulse_end = time.time()

    # Convert time to distance
    # Speed of sound = 34300 cm/s, divide by 2 (round trip)
    pulse_duration = pulse_end - pulse_start
    distance = pulse_duration * 17150
    return round(distance, 2)

def get_average_distance(samples=5):
    """Take multiple readings and average for accuracy."""
    readings = []
    for _ in range(samples):
        readings.append(get_distance())
        time.sleep(0.05)
    return sum(readings) / len(readings)

```

TRIG is set as an **OUTPUT** — the Pi sends a short pulse out to the sensor to trigger a measurement.

ECHO is set as an **INPUT** — the sensor sends a signal back to the Pi and holds it HIGH for exactly as long as the sound wave was traveling.

The 10 microsecond trigger **pulse (time.sleep(0.00001))** is the required signal to start a measurement — the HC-SR04 datasheet specifies this exact duration.

pulse_duration * 17150 converts time to centimeters. This comes from: speed of sound (34,300 cm/s) divided by 2 (because the sound travels to the target AND back). So distance = time × 17,150.

get_average_distance(5) takes 5 readings and averages them. Single readings can have noise — averaging gives more stable positioning.

The voltage divider on the **ECHO** pin is required because the HC-SR04 outputs 5V but the Pi GPIO only tolerates 3.3V. Use a 1k ohm and 2k ohm resistor: HC-SR04 ECHO → 1k ohm → [GPIO 27] → 2k ohm → GND.

time.sleep(0.5) on startup lets the sensor stabilize before taking its first reading — skipping this causes inaccurate first measurements.

Mechanical Design:

Decisions:

In terms of mechanical design there are many different ways to achieve the goal. I was able to develop CAD assemblies for two different mechanical designs (Videos and descriptions of the designs can be found in the iteration section). However there are some tips that can be universally helpful despite your exact design.

Tip 1: For all points of rotation in your design add bearings. Without bearings too much friction builds up in the system that cause a multitude of issues.

Tip 2: Design EVERYTHING in CAD first. Add all screw holes, screws, bolts, motors etc into your CAD assembly. This can seem tedious and like a waste of time, but I promise it will help you avoid many other issues that would come up in fabrication that would cost you even more time. For any hardware (screws, bolts, nuts, etc) you can find detailed 3D models on the McMaster-Carr website (<https://www.mcmaster.com/>). I have also found other useful 3D models for specific motors on GrabCad (https://grabcad.com/library/software/solidworks/tag/servo?page=1&time=all_time&sort=recent)

Tip 3: Plan for delays in fabrication and integration. It is crucial to understand the timeframe of the fabrication method you are choosing. The laser cutters and 3D printers are relatively quick ways to fabricate your design, but sometimes there can be issues. Issues with the machines hardware themselves can cause delays that are unexpected. There can be many errors in the fabrication process that can make things take much longer than you expect. Here is an example of how this can cause an issue, imagine you have your CDR in 3 days and you need to 3D print a few parts before the review. You have been told that 3D Printers are quick and easy to use so you wait till the night before your presentation to print the parts. When you come back in the morning

before your CDR to pick up your part you find a stringy mess of a print, the first layer fails to stick and the whole print is ruined, with no time to reprint before the CDR. It is crucial when you are fabricating a part that you plan to have to make multiple iterations and plan to run into errors so that you can be successful in meeting the deadlines.

Iterations:

For our initial iteration of the mechanical design we designed a system that used a set of trap doors each controlled by a servo and a rack and pinion to move an arm to push the items. The idea was that based on what item was identified by the camera, the servo would open the bin that corresponds to the category (i.e. cardboard, metal, plastic, trash). Then the rack and pinion would spin using a DC motor to have an arm come and sweep the trash into the open bin. This design proved difficult to effectively fabricate, as the bins could only be made on a smaller scale due to the size limitations on the PRUSA MK4S 3D printers. Many of the connection points came out too flimsy and not strong enough.

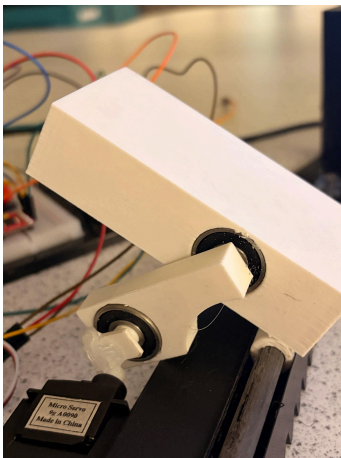
(<https://drive.google.com/file/d/1vNjZkF40G0StQ9daarqqlHj5I1OiCa3Z/view?usp=sharing>)

For our second iteration we moved to a different mechanical system. This one had a plate, controlled by a single servo that can tilt the plate up to dump off whatever item is on top of it. This plate is adhered to the rack and pinion, so that when the item is placed on the plate, the software will identify where it needs to be sorted, then the rack and pinion move the plate in front of the corresponding trash bin and the servo activates dumping the trash into that bin

The first iteration of this design did not work out well, all the connection points were PLA on PLA and things did not adhere well or spin well. So we redesigned it so that each of the connection points

connects with a bearing. This made it so that the parts could be better adhered down and have very free and frictionless rotation

I would highly recommend designing so that each connection point fits together with a bearing, unless the part needs to be completely fixed down. The bearing allows you to adhere the bearing down to both connection points securely, while still allowing for free rotation, trust me it will make your life easier. The figure shown below is an example of using bearing connections on my prototype.



This iteration is the one we have chosen to stick with, it took about 4 iterations to get the tolerances on everything correct to properly fit together. Errors with your CAD and the 3D printing WILL come up, don't expect everything to fit together perfectly on the first try, they won't. So start iterating as soon as possible and plan to have to redo many prints. This also means that you might go through your \$10 printing credit, so having multiple students on the team doing the 3D printing workshop so you can use multiple people's credit is crucial.

(https://drive.google.com/file/d/1Z8PglRCvNiXz9MyrVAHXiCI_XIrdW5mZ/view?usp=sharing)

System Integration:

The main thing that made troubleshooting simple was testing each subsystem independently before running the full integration. We had 5 isolated test scripts. [Find Scripts all below:](#)

Script name	Expected result	Common failures
TestServo.py	Servo opens then closes according to an inputted angle value	Nothing moves → check that the pin wiring is correct and that the code declares the servo's associated BCM pin
TestMotor.py	Motor moves to each bin position and returns	Nothing moves → check STBY, AIN1/2, battery wiring. Check that batteries aren't dead. Check that the code declares the motor's associated BCM pin
TestButtonCam.py	Button press captures photo and prints classification	ModuleNotFoundError → PYTHONPATH not set
FullIntegration.py	Full cycle check: button, classify, motor, servo	Drive mounts as non corresponding name → check mount name using lsblk, update compilation path

[System in Action Video](#)

Common Struggles and Debugging:

1. SD Card full during pip install

Problem: students run out of SD card storage while installing ultralytics because pip downloads GPU-targeted PyTorch packages

Solution: redirect all temp/cache directories to an external drive using TMPDIR and PIP_CACHE_DIR environment variables before running pip install

Helpful Resources:

- PyTorch CPU-only install guide: <https://pytorch.org/get-started/locally/>
- pip --target flag docs: https://pip.pypa.io/en/stable/cli/pip_install/#cmdoption-t
- Raspberry Pi storage management: <https://www.raspberrypi.com/documentation/computers/os.html>

2. ExFAT Drive Blocks venv Creation

Problem: Students try to create a Python virtual environment on an ExFAT external drive and get “operation not permitted:lib → lib64” because ExFAT doesn’t support symlinks

Solution: Keep the venv on the SD card. Install packages to the external drive using pip install --target=/path/to/drive/packages. Run scripts with PYTHONPATH=/path/to/drive/packages python3 script.py.

Helpful Resources:

- Python venv documentation: <https://docs.python.org/3/library/venv.html>
- Symlink explanation:
<https://cambass.medium.com/understanding-symlinks-and-a-common-use-case-for-linux-4013cd0e695d>
- pip --target documentation: https://pip.pypa.io/en/stable/cli/pip_install/#cmdoption-t

3. External drive mounts under different name each boot

Problem: the drive mounts as ExtraSpace on one boot and ExtraSpace2 on the next causing a failure in compiling, breaking the PYTHONPATH and causing ModuleNotFoundError

Solution: check the name of the mount using command lsblk and recompile with this in the path

Helpful Resources:

- ExFAT support on Pi: <https://pimylifeup.com/raspberry-pi-exfat/>

4. Typo in systemd Service File (Enviroment vs Environment)

Problem: a common typo causes the PYTHONPATH to never be set, resulting in ModuleNotFoundError every time service starts.

Solution: double check spelling

5. Roboflow Data set Download Fails

Problem: browser based zip downloads from Roboflow Universe silently fail or download a zip containing unrelated files (STL files, wrong format instead of images)

Solution: use the roboflow python api or switch to kaggle. Verify the dataset is classification not object detection before downloading

Helpful Resources:

- Roboflow Python API docs: <https://docs.roboflow.com/api-reference/install-python-package>
- Roboflow Universe search filters: <https://universe.roboflow.com>
- Kaggle dataset search: <https://www.kaggle.com/datasets>
- YOLOv8 dataset format guide: <https://docs.ultralytics.com/datasets/classify/>

Student Q&A Reference:

Q1. Why use a pi instead of an arduino?

A1. The Pi runs a full linux operating system and can execute Python with machine learning libraries—which Arduino can't do natively.

Resources:

- Raspberry Pi vs Arduino comparison: <https://www.raspberrypi.com/documentation>
- RPi GPIO tutorial: <https://gpiozero.readthedocs.io/en/stable/>

Q2. Why is my confidence 70% and why does it matter?

A2. Everytime the model classifies an image it outputs a probability for each class—cardboard, glass, metal, etc. The probabilities sum to 100%. Without the threshold, the model would always sort, the system would sort even with inaccurate results.

Q3. What is a virtual environment and why do I need it?

A3: A virtual environment is an isolated Python workspace with its own set of installed packages, completely separated from the system's python. On modern RPI OS, the operating system itself uses python internally and protects it from outside pip installs with a policy called “externally managed environment”. Without a venv every pip install attempt will be blocked.

Resources:

- Python venv official docs: <https://docs.python.org/3/library/venv.html>
- Raspberry Pi Python environment guide: <https://www.raspberrypi.com/documentation/computers/os.html>

Scripts:

I recommend telling students to create each testing script for every component that way you can isolate issues systematically.

ServoTest.py → compile with python3 ServoTest.py

```
import RPi.GPIO as GPIO
import time

SERVO_PIN = 18 # BCM GPIO 18 = Physical Pin 12

GPIO.setmode(GPIO.BCM)
GPIO.setup(SERVO_PIN, GPIO.OUT)
servo = GPIO.PWM(SERVO_PIN, 50)
servo.start(0)

try:
    print("Opening door...")
    servo.ChangeDutyCycle(7.5) # open position
    time.sleep(2)

    print("Closing door...")
    servo.ChangeDutyCycle(2.5) # closed position
    time.sleep(1)
    servo.ChangeDutyCycle(0) # stop signal

    print("Servo test PASSED!")

except KeyboardInterrupt:
    print("Test interrupted.")

finally:
    servo.stop()
    GPIO.cleanup()
```

MotorTest.py → compile with python3 MotorTest.py

```
import RPi.GPIO as GPIO
import time

AIN1, AIN2, PWMA = 5, 23, 13
BIN_ORDER = ['cardboard', 'glass', 'metal', 'paper', 'plastic', 'trash']
STEP_TIME = 2

GPIO.setmode(GPIO.BCM)
for pin in [AIN1, AIN2, PWMA]:
    GPIO.setup(pin, GPIO.OUT)
motor_pwm = GPIO.PWM(PWMA, 1000)
motor_pwm.start(0)

def motor_forward(duration):
    GPIO.output(AIN1, GPIO.HIGH)
    GPIO.output(AIN2, GPIO.LOW)
    motor_pwm.ChangeDutyCycle(75)
    time.sleep(duration)
    motor_stop()

def motor_backward(duration):
    GPIO.output(AIN1, GPIO.LOW)
    GPIO.output(AIN2, GPIO.HIGH)
    motor_pwm.ChangeDutyCycle(75)
    time.sleep(duration)
    motor_stop()

def motor_stop():
    motor_pwm.ChangeDutyCycle(0)
    GPIO.output(AIN1, GPIO.LOW)
    GPIO.output(AIN2, GPIO.LOW)

try:
    print("Testing each bin position...\n")
    for i, bin_name in enumerate(BIN_ORDER):
        steps = i + 1
        duration = STEP_TIME * steps
        print(f'Moving to {bin_name} ({duration}s forward)...')
        motor_forward(duration)
        time.sleep(0.5)
        print(f'Returning home ({duration}s backward)...')
```

```

        motor_backward(duration)
        time.sleep(1)
        print(f'{bin_name} PASSED!\n')
    print("All bin positions PASSED!")

except KeyboardInterrupt:
    motor_stop()

finally:
    motor_pwm.stop()
    GPIO.cleanup()

```

TestButtonCam.py → compile with
PYTHONPATH=/media/<your_username>/ExtraSpace/packages python3 TestButtonCam.py

```

from ultralytics import YOLO
from picamera2 import Picamera2
import RPi.GPIO as GPIO
from datetime import datetime
import time, os

BUTTON_PIN = 12 # BCM GPIO 12 = Physical Pin 32
SAVE_DIR = 'captures'
os.makedirs(SAVE_DIR, exist_ok=True)

GPIO.setmode(GPIO.BCM)
GPIO.setup(BUTTON_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)

print("Loading model...")
model = YOLO('best.pt')
print("Model loaded!\n")

picam2 = Picamera2()
picam2.configure(picam2.create_still_configuration())
picam2.start()
time.sleep(2)

print("Press button to scan...\n")

try:
    while True:
        if GPIO.input(BUTTON_PIN) == GPIO.LOW:

```

```

print("Button pressed! Taking photo...")
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
filename = f'{SAVE_DIR}/capture_{timestamp}.jpg'
picam2.capture_file(filename)

results = model(filename, verbose=False)
for r in results:
    label = r.names[r.probs.top1]
    confidence = r.probs.top1conf.item()
    print(f'\nPREDICTED: {label}')
    print(f'CONFIDENCE: {confidence:.2%}\n')

    probs = r.probs.data.tolist()
    ranked = sorted(zip(r.names.values(), probs),
                    key=lambda x: x[1], reverse=True)
    for name, prob in ranked:
        bar = "#" * int(prob * 30)
        print(f' {name:12s} {prob:6.2%} {bar}')

while GPIO.input(BUTTON_PIN) == GPIO.LOW:
    time.sleep(0.1)

time.sleep(0.05)

except KeyboardInterrupt:
    print("Test stopped.")

finally:
    picam2.stop()
    GPIO.cleanup()

```

UltrasonicTest.py → compile with python3 UltrasonicTest.py

```

import RPi.GPIO as GPIO
import time

# — PIN ASSIGNMENTS (BCM GPIO) —
TRIG = 17 # BCM GPIO 17 = Physical Pin 11
ECHO = 27 # BCM GPIO 27 = Physical Pin 13 — NEEDS VOLTAGE DIVIDER

```

```

GPIO.setmode(GPIO.BCM)
GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)
GPIO.output(TRIG, GPIO.LOW)

print("Letting sensor settle...")
time.sleep(0.5) # allow sensor to stabilize before first reading
print("Sensor ready! Measuring distance every 0.5 seconds.")
print("Move an object in front of the sensor to test.")
print("Press Ctrl+C to stop.\n")

def get_distance():
    """Measure distance from sensor to nearest object in cm."""
    # Send 10 microsecond trigger pulse
    GPIO.output(TRIG, GPIO.HIGH)
    time.sleep(0.00001)
    GPIO.output(TRIG, GPIO.LOW)

    # Wait for ECHO to go HIGH (sound wave sent)
    pulse_start = time.time()
    while GPIO.input(ECHO) == GPIO.LOW:
        pulse_start = time.time()

    # Wait for ECHO to go LOW (sound wave received back)
    pulse_end = time.time()
    while GPIO.input(ECHO) == GPIO.HIGH:
        pulse_end = time.time()

    # Calculate distance
    # Speed of sound = 34,300 cm/s
    # Divide by 2 because sound travels TO object and BACK
    pulse_duration = pulse_end - pulse_start
    distance = pulse_duration * 17150
    return round(distance, 2)

def get_average_distance(samples=5):
    """Take multiple readings and average for stability."""
    readings = []
    for _ in range(samples):
        readings.append(get_distance())
        time.sleep(0.05)
    return round(sum(readings) / len(readings), 2)

try:

```

```

while True:
    # Single reading
    single = get_distance()

    # Averaged reading (more stable)
    averaged = get_average_distance()

    print(f"Single reading: {single:.1f} cm")
    print(f"Averaged reading: {averaged:.1f} cm")

    # Check against bin distances
    bin_distances = {
        'cardboard': 80,
        'glass': 70,
        'metal': 60,
        'paper': 50,
        'plastic': 40,
        'trash': 30,
    }
    for material, target in bin_distances.items():
        if abs(averaged - target) <= 2.0:
            print(f">>> Platform is at {material} bin position!")
            break

    print()
    time.sleep(0.5)

except KeyboardInterrupt:
    print("Test stopped.")

finally:
    GPIO.cleanup()

```

FullIntegration.py → **compile with** → **compile with**
PYTHONPATH=/media/<your_username>/ExtraSpace/packages python3 FullIntegration.py

```

from ultralytics import YOLO
from picamera2 import Picamera2
import RPi.GPIO as GPIO
from datetime import datetime

```

```

import time, os

# — PINS —
SERVO_PIN = 18
AIN1, AIN2, PWMA = 5, 23, 13
BUTTON_PIN = 12
TRIG, ECHO = 17, 27

# — CONFIG —
BIN_DISTANCES = {
    'cardboard': 80, 'glass': 70, 'metal': 60,
    'paper': 50, 'plastic': 40, 'trash': 30
}
HOME_DISTANCE = 80
DISTANCE_TOLERANCE = 2.0
CONFIDENCE_THRESHOLD = 0.70
SAVE_DIR = 'captures'
os.makedirs(SAVE_DIR, exist_ok=True)

# — GPIO SETUP —
GPIO.setmode(GPIO.BCM)
GPIO.setup(SERVO_PIN, GPIO.OUT)
servo = GPIO.PWM(SERVO_PIN, 50)
servo.start(0)

for pin in [AIN1, AIN2, PWMA]:
    GPIO.setup(pin, GPIO.OUT)
motor_pwm = GPIO.PWM(PWMA, 1000)
motor_pwm.start(0)

GPIO.setup(BUTTON_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)
GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)
GPIO.output(TRIG, GPIO.LOW)
time.sleep(0.5)

# — ULTRASONIC —
def get_distance():
    GPIO.output(TRIG, GPIO.HIGH)
    time.sleep(0.00001)
    GPIO.output(TRIG, GPIO.LOW)
    pulse_start = time.time()
    while GPIO.input(ECHO) == GPIO.LOW:
        pulse_start = time.time()

```

```

pulse_end = time.time()
while GPIO.input(ECHO) == GPIO.HIGH:
    pulse_end = time.time()
return round((pulse_end - pulse_start) * 17150, 2)

def get_average_distance(samples=5):
    readings = [get_distance() for _ in range(samples)]
    time.sleep(0.05)
    return sum(readings) / len(readings)

# — MOTOR —
def motor_forward():
    GPIO.output(AIN1, GPIO.HIGH)
    GPIO.output(AIN2, GPIO.LOW)
    motor_pwm.ChangeDutyCycle(60)

def motor_backward():
    GPIO.output(AIN1, GPIO.LOW)
    GPIO.output(AIN2, GPIO.HIGH)
    motor_pwm.ChangeDutyCycle(60)

def motor_stop():
    motor_pwm.ChangeDutyCycle(0)
    GPIO.output(AIN1, GPIO.LOW)
    GPIO.output(AIN2, GPIO.LOW)

# — SERVO —
def door_open():
    servo.ChangeDutyCycle(7.5)
    time.sleep(1)

def door_close():
    servo.ChangeDutyCycle(2.5)
    time.sleep(1)
    servo.ChangeDutyCycle(0)

# — NAVIGATION —
current_position = 0

def go_to_position(target_cm, label='position'):
    timeout = time.time() + 20
    print(f'Moving to {label} ({target_cm}cm)...')
    while True:
        current = get_average_distance()

```

```

    error = current - target_cm
    print(f" Current: {current:.1f}cm | Target: {target_cm}cm")
    if abs(error) <= DISTANCE_TOLERANCE:
        motor_stop()
        print(f" Reached {label}!\n")
        break
    elif error > 0:
        motor_forward()
    else:
        motor_backward()
    if time.time() > timeout:
        motor_stop()
        print(f" WARNING: Timeout. Stopping motor.")
        break
    time.sleep(0.05)

def go_to_bin(label):
    go_to_position(BIN_DISTANCES[label], label)

def return_home():
    go_to_position(HOME_DISTANCE, 'home')

def sort_item(label):
    print(f"\nSorting: {label}")
    go_to_bin(label)
    door_open()
    time.sleep(2)
    door_close()
    return_home()

# — LOAD MODEL + CAMERA —
print("Loading model...")
model = YOLO('best.pt')
print("Model loaded!")

picam2 = Picamera2()
picam2.configure(picam2.create_still_configuration())
picam2.start()
time.sleep(2)
print("Camera ready!\n")

door_close()
print("Ready — press button to scan.\n")

```

```

# — MAIN LOOP —
try:
    while True:
        if GPIO.input(BUTTON_PIN) == GPIO.LOW:
            print("Button pressed! Scanning...")
            time.sleep(0.3)

            timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
            filename = f'{SAVE_DIR}/capture_{timestamp}.jpg'
            picam2.capture_file(filename)

            results = model(filename, verbose=False)
            for r in results:
                label = r.names[r.probs.top1]
                confidence = r.probs.top1conf.item()
                print(f'Predicted: {label} ({confidence:.2%})')

                if confidence >= CONFIDENCE_THRESHOLD:
                    sort_item(label)
                else:
                    print("Confidence too low — defaulting to trash.")
                    sort_item('trash')

            while GPIO.input(BUTTON_PIN) == GPIO.LOW:
                time.sleep(0.1)

            time.sleep(0.05)

except KeyboardInterrupt:
    print("\nShutting down...")
    return_home()

finally:
    picam2.stop()
    servo.stop()
    motor_pwm.stop()
    GPIO.cleanup()
    print("Done!")

```

